Introduction to Computer Systems

Lecture 5: Practical Skills in Computer Systems

Instructors: Wei Xu, Mingyu Gao
Teaching Assistants: Weitao Wan, Tingrui Zhang, Ruitao Liu, Shuo Chen

April 7, 2026

Contents

1 Introduction **3**

2 The Unix Shell as a Programmable Interface **3**

2.1 From system calls to human-usable interaction . . . 3

2.2 The Unix philosophy: composition over monoliths . . . 4

2.3 Why command lines matter again in the AI era . . . 4

3 Core Commands, Identity, and the Notion of Privilege **4**

3.1 Basic orientation commands . . . 4

3.2 Users, UID 0, and the `root` account . . . 6

3.3 `sudo` and least privilege . . . 6

4 Files, Directories, Paths, and Permissions **7**

4.1 Directories are namespaces, not just folders . . . 7

4.2 Absolute and relative paths . . . 7

4.3 Reading files efficiently . . . 7

4.4 Long directory listings and hidden files . . . 8

4.5 Permission bits and octal notation . . . 8

5 Processes, Environment Variables, and Standard Streams **9**

5.1 Processes as running programs . . . 9

5.2 Standard input, output, and error . . . 9

5.3 Why multiple processes print into one terminal . . . 10

5.4 Environment variables . . . 10

5.5 Inspecting processes with `ps` and `top` . . . 10

6 Composition: Exit Codes, Redirection, and Pipes **11**

6.1 Sequencing commands with success and failure . . . 11

6.2 Special shell variables . . . 11

6.3 Redirection operators . . . 12

6.4 Pipes as inter-process communication . . . 12

6.5 `/dev/null`, command substitution, and `tee` . . . 14

7 Interactive Efficiency, Signals, and Job Control **14**

7.1 Terminal shortcuts . . . 14

7.2 Signals and process control . . . 14

7.3 Foreground, background, and shell jobs . . . 15

7.4 `nohup`, disconnections, and `tmux` . . . 15

8 Pattern Expansion, Logs, and Regular Expressions **16**

8.1 Shell globbing versus regular expressions . . . 16

8.2 Where to look when something goes wrong . . . 16

8.3 Regular expressions as a computational model . . . 17

9 The Unix Data-Processing Toolbox **17**

9.1 Learning commands with `man` and `tldr` . . . 17

9.2 Finding files with `find` . . . 18

9.3 Sorting and deduplication . . . 18

9.4 `sed`: noninteractive stream editing . . . 18

9.5 `awk`: pattern-action programming over text tables . . . 19

10 Remote Access, File Transfer, and Streaming Data Across Machines **19**

10.1 SSH as the standard remote shell . . . 19

10.2 Moving files efficiently . . . 19

10.3 Operational hygiene for remote work . . . 20

11 Build Automation with `make` **20**

11.1 Building is not the same as compiling . . . 20

11.2 Targets, prerequisites, and recipes . . . 20

11.3 Variables, clean targets, and repeated LaTeX builds . . . 21

12 Summary **22**

13 Quizzes & Exercises **23**

13.1 Short-Answer Questions . . . 23

13.2 Essay Questions . . . 23

13.3 Selected Answer Sketches . . . 24

1 Introduction

This lecture occupies a distinctive place in an introductory computer-systems course. Earlier lectures in such a course typically emphasize mechanisms: processes, virtual memory, caches, storage, protection, or machine organization. By contrast, this lecture is about *practical leverage*. It studies the command-line environment not as an antiquated interface, but as a programmable systems substrate that makes a programmer faster, more precise, and better able to reason about what a computer is actually doing.

A central pedagogical point is that many students encounter Unix-like systems only intermittently. A modern student may grow up primarily with graphical interfaces on Windows or macOS and therefore miss the design logic of the shell entirely. Yet engineers routinely assume familiarity with commands such as `ls`, `grep`, `ps`, `ssh`, `make`, or `docker`. The resulting knowledge gap is not merely about syntax; it is about systems literacy. A student who cannot read a shell command cannot comfortably read documentation, debug remote systems, or evaluate whether an AI-generated command is safe to execute.

This note therefore expands the lecture into a coherent set of principles. The goal is not to memorize every flag of every command. The real goal is to understand why the Unix shell is designed the way it is, how files and processes interact through inherited descriptors and environment variables, why composition through streams is so powerful, and how a family of small tools can scale from everyday convenience to serious systems work.

Definition

The **shell** is a user-level program that accepts textual commands, interprets them according to a command language, launches other programs, manages their input and output streams, and provides scripting facilities for composing multiple commands into larger workflows.

In the age of AI-assisted programming, command-line interfaces have become attractive again because they expose a stable, text-based, automatable contract. A graphical user interface requires interpretation through vision and control through clicking. A shell command, by contrast, is already a structured, machine-readable instruction. The more a system can be driven through commands, the easier it is to automate, inspect, reproduce, and verify.

2 The Unix Shell as a Programmable Interface

2.1 From system calls to human-usable interaction

At the lowest level, the operating system exposes services through *system calls*. These calls are designed for programs, traditionally C programs, and therefore use machine-oriented representations: integers, byte buffers, pointers, flags, and structured conventions. Human users do not interact naturally in that form. A shell exists to bridge this gap. It provides a higher-level textual layer that turns strings typed by a user into operations the operating system can understand.

The important conceptual refinement is that Unix shells do not merely translate one command at a time. They constitute a programming language. This is the decisive design difference between a minimal command launcher and a shell in the classical Unix sense. Once a user can combine commands, redirect outputs, assign variables, branch, loop, and define functions, the shell becomes a genuine programming environment.

Deep Dive

The shell is historically significant because it pushes programmability all the way to the user interface. Instead of making the operating system accessible only through compiled programs, Unix makes the interactive interface itself scriptable. This design decision is one reason Unix environments remain disproportionately influential in systems work, cloud operations, and modern development tooling.

2.2 The Unix philosophy: composition over monoliths

The classical Unix philosophy is often summarized as follows: build simple tools, let each tool do one thing well, and make them easy to connect. This is not just aesthetic minimalism. It is an engineering response to complexity. A single command may do something seemingly trivial, such as printing a string with `echo`. By itself this is unremarkable. But when such tools are connected into pipelines and scripts, they form surprisingly expressive programs.

The shell operates naturally on strings and lists of strings. This is both its strength and one of its major limitations. The strength is composability: the output of one command can become the input of another with almost no friction. The limitation is fragility when data contain spaces, newlines, or other characters that interact badly with naive string processing. Much of advanced shell use therefore involves disciplined quoting, expansion control, and an awareness of how the shell tokenizes input.

2.3 Why command lines matter again in the AI era

A command-line interface is especially well suited to automation. It is deterministic, textual, and explicit. An agent does not need to infer button locations from pixels or guess how a menu hierarchy works. It only needs to emit a command in the right form and inspect the resulting text. See Figure 1.

This does not mean graphical interfaces are unimportant. For many human tasks, they remain easier to learn and more approachable. But from a systems perspective, a textual interface offers stable semantics. It can be stored in shell history, reviewed in code review, copied into documentation, embedded in scripts, and generated by other programs. This is precisely why understanding shell commands is now valuable even to us who may eventually rely heavily on AI assistants: one must be able to inspect and judge what the assistant proposes.

3 Core Commands, Identity, and the Notion of Privilege

3.1 Basic orientation commands

A small set of commands provides immediate situational awareness in a shell session. Table 1 summarizes the commands emphasized in the lecture.

The command `time` deserves special emphasis because some people often misread it as a clock query. Its output usually includes three metrics:

- `real`: wall-clock elapsed time;
- `user`: CPU time spent executing program instructions in user mode;

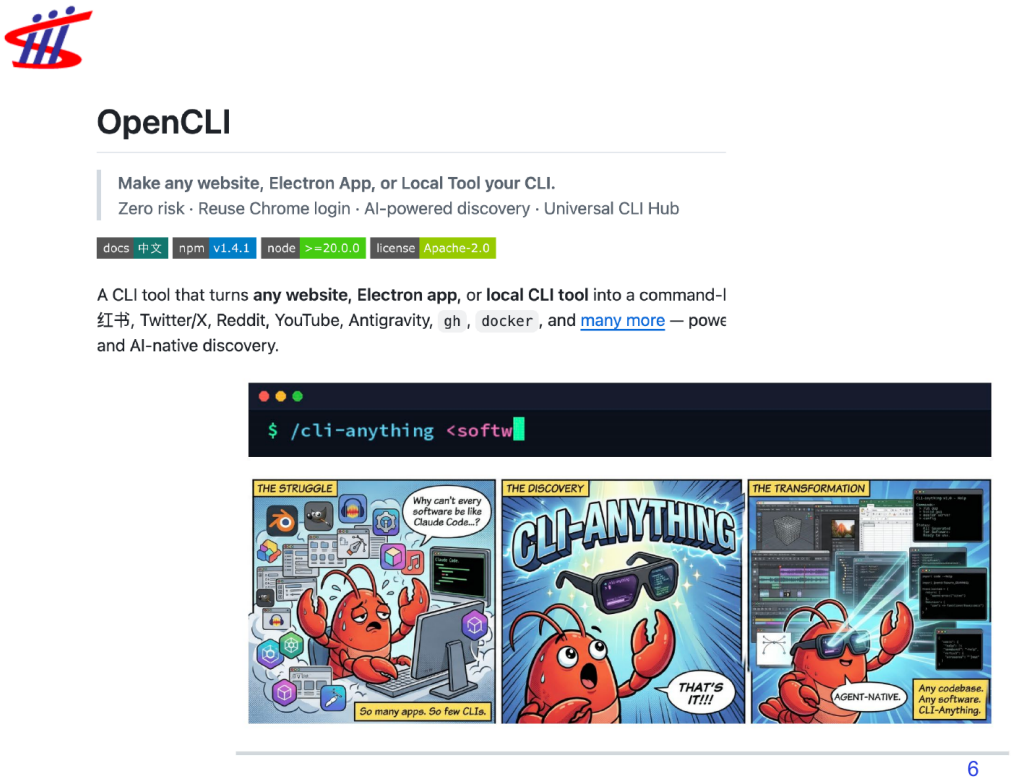


Figure 1: A contemporary trend: wrapping graphical or web-based tools with command-line interfaces so that both humans and automated agents can drive them through text. The pedagogical point is not the specific project name, but the systems lesson that textual interfaces are easier to script, inspect, version, and compose than click-driven workflows.

Table 1: Core introductory shell commands and their systems significance.

Command	Purpose and systems interpretation
`date`	Print the current date and time. Useful for timestamping logs, sanity-checking clock configuration, and confirming time synchronization issues.
`time cmd`	Measure the runtime of a command. The reported `real`, `user`, and `sys` fields help distinguish wall-clock delay from CPU time spent in user mode or kernel mode.
`uptime`	Show how long the machine has been running and report the load average. Useful for quick operational checks, including after a reboot.
`man cmd`	View the manual page for a command. Historically central to self-service documentation in Unix systems.
`uname`	Print system identification information, such as kernel or operating-system name. It does *not* print the current username.
`whoami`	Print the current effective username. Important when debugging permission problems or remote sessions.
`clear`	Clear the terminal display. This is convenience rather than mechanism, but it matters in interactive work.

- `sys`: CPU time spent in the kernel on behalf of the program.

A compute-bound task typically shows substantial `user` time. An I/O-heavy task may show comparatively more `sys` time because the kernel is doing more work related to file operations, buffering, or networking. On multicore systems, `user` and `sys` time can even sum to more than `real` time because CPU time is aggregated across cores.

The command `uptime` also rewards interpretation. The load average is not a percentage; it is roughly an exponentially weighted average of the number of runnable or uninterruptible tasks over recent windows, commonly one, five, and fifteen minutes. A load of `4.0` means something very different on a single-core machine than on a 16-core server.

3.2 Users, UID 0, and the `root` account

Unix systems distinguish users explicitly and rely on this distinction for both administration and protection. The account `root` is special because its user identifier is 0. In most Unix-like systems, UID 0 implies unrestricted privilege checks at the kernel level. This account can read and modify system files, kill arbitrary processes, mount filesystems, reconfigure networking, and so on.

Definition

The **root user** is the superuser account on Unix-like systems. Conventionally, any process running with effective UID 0 bypasses many ordinary permission checks and therefore has full administrative power over the system.

This design is simple and powerful, but dangerous. A shell running as `root` can delete or rewrite critical files with no further confirmation. Consequently, good operational practice is to spend as little time as possible logged in as `root` directly.

3.3 `sudo` and least privilege

The command `sudo` allows an authorized user (i.e. users in "sudoer" file) to run a specific command with elevated privileg (using his password rather than root's password), usually as `root`. This is preferable to working permanently in a privileged shell because it localizes risk. A user can remain in an unprivileged context for ordinary work and escalate only for specific administrative operations.

Deep Dive

There is an important conceptual distinction between **safety** and **security**. Safety is protection against mistakes made by legitimate users, including one's future self. Security is protection against malicious behavior or unauthorized access. The Unix privilege model supports both: it stops users from casually damaging system state, and it also constrains attackers who do not already possess elevated privilege.

Many production systems disable direct remote login as `root`. The reason is straightforward: every Unix attacker knows the username `root`. If direct root login is disabled, an attacker must guess not only a password or key but also a valid ordinary username before privilege escalation is even possible.

4 Files, Directories, Paths, and Permissions

4.1 Directories are namespaces, not just folders

Commands such as `ls`, `cd`, `mkdir`, and `pwd` form the basic vocabulary of file navigation. However, the underlying concept is more subtle than the graphical metaphor of a folder. A directory is a mapping from names to filesystem objects. The namespace often looks like a tree, but once symbolic links are considered, the visible structure becomes a graph rather than a strict tree.

This matters whenever programs recursively traverse directory hierarchies. A naive traversal algorithm that assumes there are no cycles can revisit the same location indefinitely through symbolic links. Systems software must therefore either avoid following symlinks blindly or track visited locations carefully.

4.2 Absolute and relative paths

The working directory, reported by `pwd`, is the reference point for relative paths. The special names `.`, `..`, and `~` refer respectively to the current directory, the parent directory, and the current user's home directory.

A particularly instructive case is executing a program in the current directory. If a compiled program is named `a.out`, typing `a.out` often does not execute it. One instead writes `./a.out`. This is not arbitrary syntax. It explicitly names the current directory and avoids depending on `PATH` lookup.

Deep Dive

Most Unix-like systems do *not* include the current directory in `PATH` by default. This is a defensive design choice. If the shell always searched `.` automatically, a malicious executable with a familiar name such as `ls` or `python` could trick a user into running the wrong program simply by being placed in the current directory.

4.3 Reading files efficiently

The commands `cat`, `less`, `head`, and `tail` support different modes of reading.

- `cat` sends file contents directly to standard output;
- `less` and `more` page through data interactively;
- `head` and `tail` show only the beginning or end of a file.

For systems work, `tail -f` is especially important because logs are often appended over time. Rather than stare at a terminal while a long-running job emits mixed output, it is often better to redirect that output into a file and watch the file with `tail -f`. This preserves a durable record and reduces the risk that important messages scroll out of view.

The commands `wc -l` and `df -kh` illustrate two further Unix habits. The former counts lines quickly without requiring full visual inspection. The latter reports disk usage in human-readable units; the options reflect Unix's historical bias toward terse defaults and later compatibility-preserving extensions.

4.4 Long directory listings and hidden files

`ls -al` combines two important views: `-a` shows hidden entries and `-l` shows metadata in long format. In Unix, a file is hidden simply because its name begins with a dot. This is a naming convention, not a separate attribute bit in the style of older desktop systems.

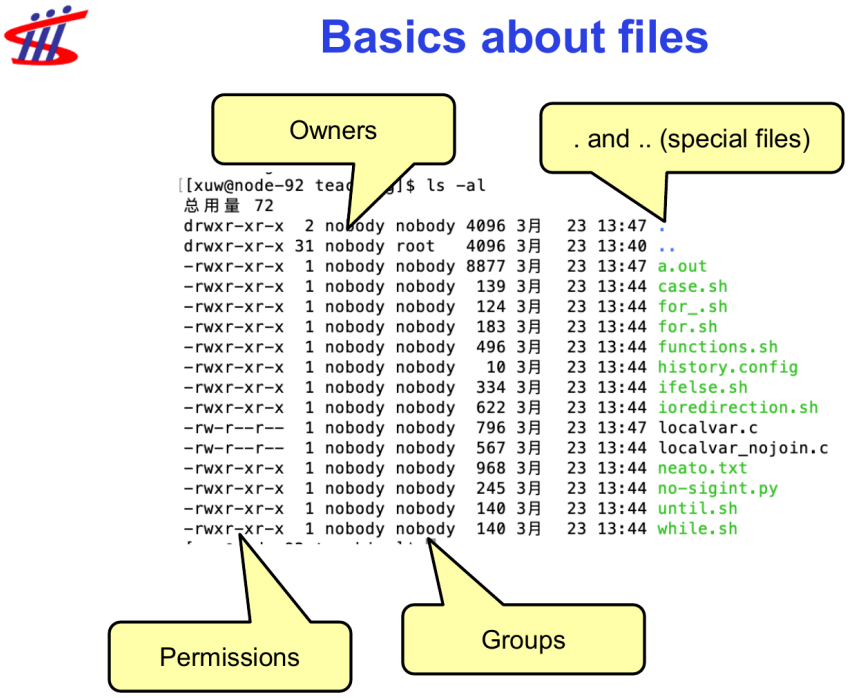


Figure 2: Annotated long-format directory. The figure highlights how `ls -al` simultaneously encodes file type, permission bits, owner, group, size, timestamp, and filename, while also exposing the special entries `.` and `..` that anchor relative navigation in the filesystem namespace.

The first character in the mode field indicates object type, for example `d` for directory. The next nine characters are permission bits grouped into three triplets: owner, group, and others. Each triplet is interpreted as read, write, and execute.

4.5 Permission bits and octal notation

Permission strings such as `rwxr-xr-x` are often written in octal form. The standard mapping is

$$\texttt{r} = 4, \qquad \texttt{w} = 2, \qquad \texttt{x} = 1.$$

Thus `755` means owner `rwx`, group `r-x`, others `r-x`. Similarly, `644` means owner `rw-`, group `r-`, others `r-`, and `600` restricts read/write access to the owner alone.

The commands `chmod` and `chown` change permissions and ownership respectively. For example:

Listing 1: Typical permission and ownership operations.

```
chmod 755 script.sh
chmod 600 ~/.ssh/id_ed25519
chown alice:students report.txt
```

An important modern observation is that permissions protect against *other users*, not against a process already running as the owner. If an AI coding assistant is launched under your account, then any file readable by you is, in principle, readable by that process too. This is one reason privilege separation and careful tool trust matter.

5 Processes, Environment Variables, and Standard Streams

5.1 Processes as running programs

A process is a running instance of a program together with its current execution state, address space, open file descriptors, signal dispositions, and other kernel-managed metadata. Every process has a process ID (PID), and almost every process also has a parent process ID (PPID), reflecting the process hierarchy formed through creation events such as `fork()`.

The shell is central to this story because it repeatedly creates child processes to run commands. In a typical Unix execution model, the shell performs a `fork()` to create a child and then that child executes a new program image via `execve()`. The child inherits selected execution context from the parent, including open file descriptors and environment variables.

5.2 Standard input, output, and error

Every newly started process begins with three conventional streams:

- file descriptor 0: standard input (`stdin`);
- file descriptor 1: standard output (`stdout`);
- file descriptor 2: standard error (`stderr`).

These are "just files" in the Unix sense. They may refer to an actual terminal device, a regular file, a pipe, or a network socket. The abstraction is powerful precisely because the process need not care.

Definition

A **file descriptor** is a small nonnegative integer maintained by the kernel to identify an open file-like object within a process. Unix treats regular files, terminals, pipes, and sockets through a largely uniform interface built around descriptors.

The idea appears directly in systems programming:

Listing 2: Writing explicitly to standard error in C++.

```cpp
#include <iostream>

int main() {
    std::cerr << "This is an error message." << std::endl;
    return 0;
}
```

Listing 3: Writing explicitly to standard error in C.

```c
#include <stdio.h>

int main(void) {
```

```c
    fprintf(stderr, "This is an error message.\n");
    return 0;
}
```

5.3 Why multiple processes print into one terminal

Some people often ask why outputs from different commands can appear interleaved in the same terminal window. The answer is inheritance. If two child processes inherit the same file descriptor 1 and both write to it concurrently, their outputs go to the same destination. The terminal is not "owned" by one process in the everyday sense; it is simply the endpoint currently associated with particular descriptors.

This same model explains remote login. During an SSH session, the shell's standard input and output are effectively connected to a network socket rather than a local display device. Remote child processes inherit those descriptors, so their output returns across the network to the SSH client on the user's machine.

5.4 Environment variables

Environment variables are inherited name–value bindings used to parameterize a process's execution context. Common examples include:

- `PATH`: directories searched for executable commands;
- `PWD`: current working directory;
- `PS1`: shell prompt format;
- language- or toolchain-specific variables that alter library search paths, compilers, package managers, or authentication tokens.

A shell assignment is syntactically delicate:

Listing 4: Environment-variable assignment and export.

```
FOO=bar
export PATH=/home/alice/bin:$PATH
echo $PATH
```

Whitespace matters. `FOO=bar` is an assignment, but `FOO = bar` is parsed as an attempt to run a command named `FOO`. This is one example of the shell's strict token-based grammar.

Python virtual environments are a useful modern case study. Conceptually, they work largely by modifying environment and path state so that one interpreter and one set of packages are found before others. In this way, the shell's execution context becomes a practical mechanism for dependency isolation.

5.5 Inspecting processes with `ps` and `top`

The command `ps` provides snapshots of process state, while `top` provides a dynamic view of running processes and resource consumption. A common pattern is:

Listing 5: Basic process inspection in the shell.

```
ps -ef | grep alice
```

```
top
```

The pipeline is conceptually simple: list processes, then filter lines. However, it also reflects the broader Unix pattern that generic text-processing tools can serve as a process-observation language. This is typical of shell work: the interface to one subsystem becomes usable because it composes cleanly with another.

6 Composition: Exit Codes, Redirection, and Pipes

6.1 Sequencing commands with success and failure

The shell attaches semantic meaning to process exit status. By convention, an exit code of 0 indicates success; nonzero values indicate failure or other distinguished conditions. This convention enables compact command composition:

Listing 6: Command chaining based on exit status.

```
ping -c 1 example.com && wget https://example.com/file.zip
make || echo "build failed"
cmd1 ; cmd2
```

The operators have distinct meanings:

- `cmd1 && cmd2`: run `cmd2` only if `cmd1` succeeds;
- `cmd1 || cmd2`: run `cmd2` only if `cmd1` fails;
- `cmd1 ; cmd2`: run both sequentially regardless of status.

This is not merely shell convenience. It is a tiny control-flow language based on operating-system return codes.

6.2 Special shell variables

A few special variables are especially useful in interactive shell work:

- `$?`: exit code of the previous command;
- `$$`: PID of the current shell or script;
- `!!`: the previous command line.

The last of these supports a classic pattern:

Listing 7: Re-running the previous command with privilege.

```
cat /etc/shadow
sudo !!
```

This is convenient after a “permission denied” failure. However, the conceptual lesson is more important than the shortcut: shell history is itself a programmable resource.

6.3 Redirection operators

Because the first three file descriptors are standardized, the shell can redirect them with concise syntax. Table 2 summarizes the most important operators.

Table 2: Core shell redirection operators.

Form	Meaning
`>`	Redirect standard output to a file, overwriting existing contents.
`»`	Redirect standard output to a file, appending instead of overwriting.
`<`	Redirect standard input from a file.
`2>`	Redirect standard error to a file.
`2>&1`	Redirect standard error to the current destination of standard output.
`|`	Connect the standard output of one process to the standard input of another.

A few canonical examples illustrate the mechanics:

Listing 8: Basic redirection patterns.

```
echo hello > hello.txt
cat < hello.txt
cat < hello.txt > hello2.txt
grep ERROR app.log > matches.txt 2> errors.txt
```

6.4 Pipes as inter-process communication

A pipe connects the output stream of one process directly to the input stream of another. The shell syntax makes the idea look lightweight, but the underlying kernel object is a bounded in-memory buffer with synchronization semantics.

Consider the command

Listing 9: A pipeline that progressively refines data.

```
curl --head --silent baidu.com \
  | grep --ignore-case content-length \
  | cut --delimiter=' ' -f2
```

The first process fetches data, the second filters lines, and the third extracts a field. This is a miniature dataflow program. The shell encourages thinking in these stages: generate, filter, transform, and consume.

Deep Dive

Pipelines are a concrete example of **stream-oriented architecture**. Each stage only needs to understand its local contract: what it reads and what it writes. The shell then supplies the glue. This style scales surprisingly well, from one-line ad hoc commands to robust production scripts.

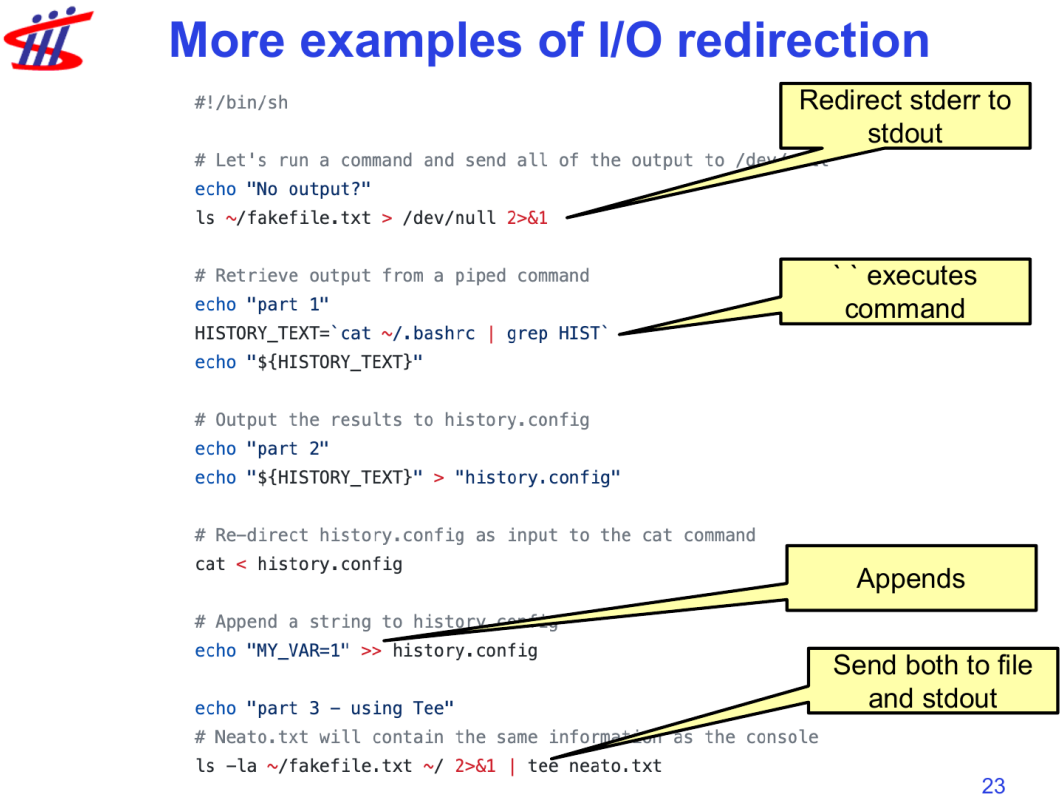


Figure 3: The figure emphasizes several central patterns: discarding output through `/dev/null`, capturing a command's output into a variable with command substitution, appending with `>>`, redirecting file contents into a command with `<`, merging standard error into standard output with `2>&1`, and duplicating a stream with `tee`.

6.5 `/dev/null`, command substitution, and `tee`

The special device `/dev/null` behaves like a sink that accepts writes and discards all data. Redirecting output to it is the shell equivalent of explicit suppression.

Command substitution, written as `$(cmd)`, evaluates a command and substitutes its standard output into the surrounding shell expression. Process substitution, often written as `<(cmd)`, materializes a command's output as a file-like object so that tools expecting filenames can still be used.

The command `tee` is another elegant piece of stream logic: it copies its standard input both to standard output and to one or more files (imagine the letter T). This is invaluable when debugging pipelines because it allows inspection without breaking the dataflow.

7 Interactive Efficiency, Signals, and Job Control

7.1 Terminal shortcuts

The shell is not only a batch environment; it is an interactive editor. A few key bindings dramatically improve efficiency:

- `Tab`: completion;
- `Ctrl-R`: reverse history search;
- `Ctrl-A`/`Ctrl-E`: jump to beginning/end of the current line;
- `Ctrl-C`: send interrupt;
- `Ctrl-Z`: suspend the foreground job;
- `Ctrl-D`: send end-of-file on input.

These bindings matter because shell use is repetitive. Engineering productivity often depends less on knowing an exotic command than on avoiding repeated retyping, avoiding mouse context switches, and preserving flow.

7.2 Signals and process control

Signals are asynchronous notifications sent to processes. Several appear repeatedly in shell work:

- `SIGINT`: typically generated by `Ctrl-C`; asks the process to interrupt;
- `SIGQUIT`: often generated by `Ctrl-\`; usually terminates and may request a core dump;
- `SIGTERM`: the default signal used by `kill`; a polite request to terminate;
- `SIGKILL`: sent by `kill -9`; cannot be caught or ignored;
- `SIGHUP`: hangup signal, traditionally associated with terminal disconnect;
- `SIGSTOP`/`SIGTSTP`: stopping signals used in job control.

A technical subtlety is worth noting: when a user presses `Ctrl-Z`, the terminal typically delivers `SIGTSTP`, not `SIGSTOP`. The distinction matters because `SIGTSTP` can be handled or ignored, while `SIGSTOP` cannot. Introductory presentations often blur the two because both are associated with stopping a process.

7.3 Foreground, background, and shell jobs

The shell tracks *jobs*, which are shell-level handles for processes or process groups launched from that shell. If a foreground process is suspended with `Ctrl-Z`, the user can inspect jobs with `jobs`, resume one in the background with `bg`, or bring one back to the foreground with `fg`.

Listing 10: Basic job-control workflow.

```
sleep 1000
# press Ctrl-Z
jobs
bg %1
kill -TERM %1
```

Background execution is useful but not magical. A process running in the background may still write to the terminal, leading to messy interleaving or blocking behavior depending on the job-control mode of the shell and terminal. In practice, long-running background jobs often redirect output explicitly.

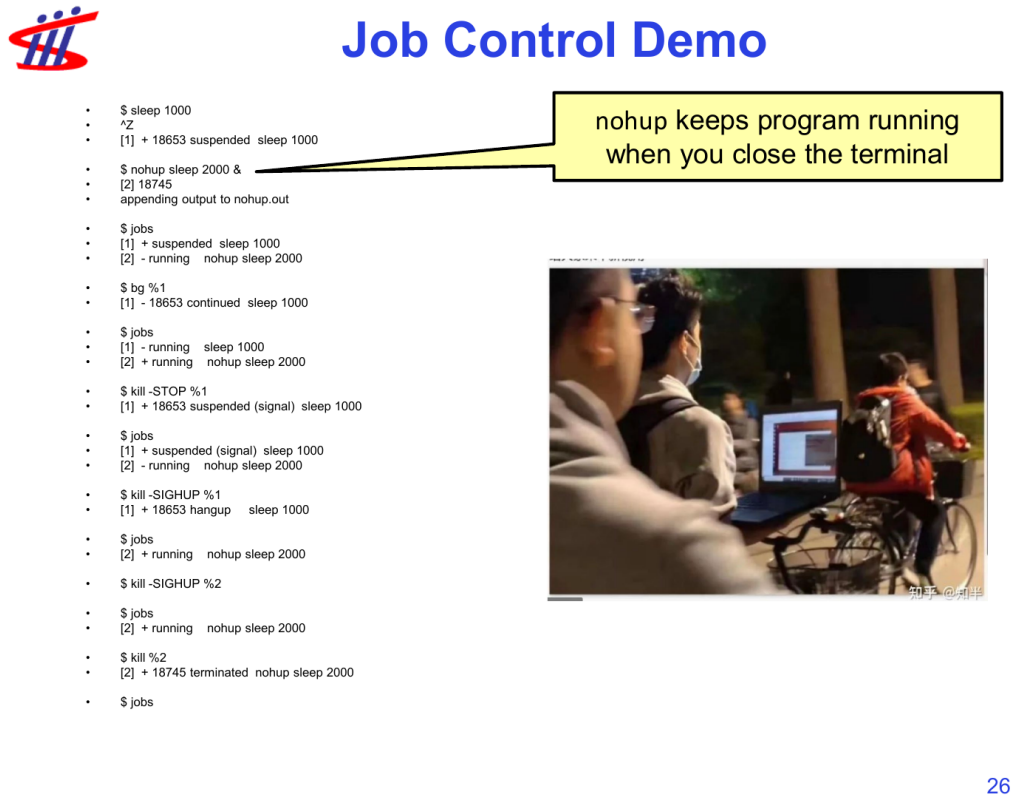


Figure 4: Job-control transcript. The sequence demonstrates suspension with `Ctrl-Z`, inspection with `jobs`, resumption via `bg`, signaling via `kill`, and the persistence effect of launching a job with `nohup`. It captures the central operational lesson that process management in the shell is a live dialogue with the kernel's process table and signal mechanism.

7.4 `nohup`, disconnections, and `tmux`

The command `nohup` tells a process to ignore `SIGHUP`. Historically this was crucial for long-running jobs launched from remote sessions, because a network disconnect could cause the controlling terminal to disappear and the attached process to terminate.

However, `nohup` solves only one problem: surviving hangup. It does not give a user a recoverable interactive workspace. Terminal multiplexers such as `screen` and especially `tmux` are more capable because they introduce a persistent virtual terminal. Processes attach to that session rather than directly to the fragile network connection. The user can detach, reconnect later, and reattach without losing the job's state.

8 Pattern Expansion, Logs, and Regular Expressions

8.1 Shell globbing versus regular expressions

Some people often confuse wildcard expansion in the shell with regular expressions. They are related only superficially. Globbing is a filename-expansion mechanism performed by the shell before a command runs. Regular expressions are a pattern language used by tools such as `grep`, `sed`, `awk`, editors, and programming languages.

Table 3: Shell globbing and regular expressions are not the same abstraction.

Aspect	Shell globbing	Regular expressions
Primary use	Expanding filenames before command execution	Matching structured text within programs or tools
Typical symbols	`*`, `?`, brace expansion such as `{png,jpg}`	`.`, `*`, `+`, character classes, anchors, grouping, alternation
Where evaluated	By the shell itself	By regex-aware programs such as `grep`, `sed`, `awk`, Python, etc.
Common pitfall	Accidentally matching too many files	Misunderstanding operator precedence or escaping rules

Examples of globbing:

Listing 11: Brace and wildcard expansion in the shell.

```
convert image.{png,jpg}
cp /path/to/project/{foo,bar,baz}.sh /newpath
mv *{.py,.sh} new_folder
mkdir foo bar && touch {foo,bar}/{a..h}
```

These are expanded by the shell before the target command executes. This is powerful, but dangerous if combined with destructive commands such as `rm` or `mv` without first checking the match set.

8.2 Where to look when something goes wrong

Practical systems work requires knowing where evidence accumulates. Useful diagnostic locations and commands include:

- `/var/log` and `tail -f` for log monitoring;
- `dmesg` for kernel and device-driver messages;
- `ifconfig` and `route` for classic network-state inspection;
- `top` for resource contention and runaway processes.

On modern Linux systems, `ip addr` and `ip route` often replace `ifconfig` and `route`. Nonetheless, older tools remain worth recognizing because legacy tutorials, scripts, and systems documentation still refer to them.

8.3 Regular expressions as a computational model

Regular expressions are not only a convenient notation for text matching; they also correspond, in their classical form, to the theory of regular languages and finite automata. This makes them conceptually significant in computer systems because so much operational data is textual: logs, command output, configuration files, compiler diagnostics, and protocol traces.

Definition

A **regular expression** is a formal pattern language for describing sets of strings. In systems practice, regexes are used to recognize, filter, validate, or transform text. In theory, classical regular expressions correspond to languages recognized by finite automata.

Important operators include:

- `.`: any single character (except newline in many tools);
- `*`: zero or more repetitions of the previous expression;
- `+`: one or more repetitions;
- `[abc]`: any one character from a set;
- `(A|B)`: alternation;
- `^` and `$`: beginning and end of line anchors.

Representative examples:

Listing 12: Representative regular-expression patterns.

```
gr(a|e)y
gr[ae]y
^dog
go+gle
\d{5}(-\d{4})?
[2-9]|[12]\d|3[0-6]
\d+(\.\d\d)?
```

Systems programmers often rely on AI assistants to generate or explain regexes. That is a reasonable workflow, but only if the user can still inspect the result critically. Regex syntax is compact enough that one mistaken quantifier or unescaped character can radically change the match set.

9 The Unix Data-Processing Toolbox

9.1 Learning commands with `man` and `tldr`

Manual pages are comprehensive but often dense. The `tldr` tool provides a more example-oriented front end that is particularly useful when learning by imitation. The intended workflow

is pragmatic: use `tldr` to discover common patterns quickly, then consult `man` when precise semantics matter.

A productive habit is to remember not full command reference manuals, but *which tool probably solves which class of problem*. Know that a command exists, know approximately what it is for, and know how to read documentation or ask for help when needed.

9.2 Finding files with `find`

The command `find` traverses a directory tree and selects filesystem objects based on predicates such as name, type, size, or modification time. Because it performs structured traversal rather than simple pattern expansion, it is much more expressive than shell globbing.

Listing 13: Common `find` patterns.

```
find . -name '*.py'
find . -type d -iname '*lib*'
find . -name '*.py' -not -path '*/site-packages/*'
find . -size +500k -size -10M
find . -name '*.c' -exec wc -l {} \;
find . -daystart -mtime -7 -delete
```

The last two examples explain why `find` commands deserve caution. Once `-exec` or `-delete` enters the picture, one is no longer merely searching; one is performing bulk operations that may be irreversible.

9.3 Sorting and deduplication

The command `sort` is one of the simplest and most deceptively subtle tools in Unix text processing. By default it sorts lexicographically as strings, not numerically. Thus `10` sorts before `2`. Numeric ordering requires `-n`. Column selection requires `-k`.

Listing 14: Sorting numerically by the first field.

```
sort -nk1,1 scores.txt
```

The command `uniq` removes adjacent duplicate lines only. Therefore the idiomatic deduplication pattern is

Listing 15: Why sorting usually precedes `uniq`.

```
sort words.txt | uniq
```

This is a perfect example of Unix composition: neither command alone solves general deduplication of unsorted text, but together they do.

9.4 `sed`: noninteractive stream editing

`sed` is a stream editor designed for scripted text transformations. The most common pattern is substitution:

Listing 16: Simple substitution with `sed`.

```
sed 's/old/new/' file.txt
```

9.5 awk: pattern-action programming over text tables

awk treats each input line as a record split into fields, conventionally by whitespace. The programmer writes a sequence of pattern–action rules. If the pattern matches, the action executes. Historically, the name awk is not derived from "awkward"; it comes from the surnames of its three original authors: Alfred Aho, Peter Weinberger, and Brian Kernighan (The author of famous "The C Programming Language" and "UNIX: A History and a Memoir").

Listing 17: A small awk program using field conditions and a regular expression.

```
awk '$1 == 1 && $2 ~ /^c[^ ]*e$/ { print $2 }' data.txt
```

This command says: if field 1 equals 1 and field 2 matches the regular expression, print field 2. awk can also accumulate counters, compute aggregates, and emit reformatted output. It is a compact domain-specific language for structured text.

10 Remote Access, File Transfer, and Streaming Data Across Machines

10.1 SSH as the standard remote shell

ssh provides encrypted remote login and command execution. In modern systems work it is nearly unavoidable. A user typically creates a keypair with ssh-keygen and authorizes passwordless login by appending the public key to ~/.ssh/authorized_keys on the remote host.

Listing 18: A standard public-key authorization pattern.

```
ssh-keygen
cat ~/.ssh/id_ed25519.pub | ssh user@remote 'cat >> ~/.ssh/authorized_keys'
```

This pattern is deceptively rich in systems concepts. It uses a local file, a pipe, a remote shell command, and standard output redirection on the remote side. It is a clean example of how local and remote dataflow can be unified by streams.

10.2 Moving files efficiently

Several command-line tools move data across machines:

- scp: secure copy with a familiar file-copy mental model;
- rsync: efficient synchronization that transfers only changed data;
- sshfs: mount remote filesystems over SSH;
- mosh: a resilient remote-shell alternative for unstable connections.

A particularly elegant Unix pattern avoids temporary archive files:

Listing 19: Streaming a directory archive directly over SSH.

```
tar -czf - /path/to/files | ssh user@host2 "cd /path/to/destination && tar -xzf -"
```

The local tar writes a compressed archive stream to standard output. SSH transports that stream. The remote tar reads from standard input and extracts it in place. This avoids both intermediate files and the overhead of copying many tiny files individually.

10.3 Operational hygiene for remote work

Remote development introduces two recurring concerns: survivability and reproducibility. Survivability is addressed by tools such as `tmux`, `nohup`, and reliable key-based login. Reproducibility is addressed by clean scripts, explicit dependencies, and eventually containers. These themes converge in the later sections on `make`, Bash scripting, and Docker.

11 Build Automation with `make`

11.1 Building is not the same as compiling

A compiler transforms source code, usually one translation unit at a time, into object code or another lower-level representation. Building software is broader. It includes dependency management, ordering constraints, linking, code generation, asset transformation, test execution, packaging, and cleanup. In a nontrivial project, building is therefore a graph problem: if target T depends on files A, B, and C, one must know what to rebuild and in what order.

Definition

A **Makefile** is a declarative build specification that describes targets, prerequisites, and recipes. A build tool such as `make` uses this specification, together with filesystem timestamps, to decide which parts of a project must be regenerated.

Algorithm 1: Timestamp-Based Incremental Rebuild

Input: A target t, prerequisite set $P(t)$, and file modification times
Output: An up-to-date artifact for t

foreach $p \in P(t)$ **do**
  if *p is generated by another rule* **then**
    recursively rebuild p;
if *t does not exist* **then**
  execute the recipe for t;
else
  if *there exists* $p \in P(t)$ *with* $\text{mtime}(p) > \text{mtime}(t)$ **then**
    execute the recipe for t;
return updated target t;

The beauty of this strategy is incrementalism. Large projects need not rebuild from scratch when a single source file changes.

11.2 Targets, prerequisites, and recipes

A Makefile rule has the form

Listing 20: A minimal Makefile rule.

```
program: main.o util.o
    gcc -o program main.o util.o
```

The line before the colon names the target. The items after the colon are prerequisites. The indented line is the recipe to execute. The indentation must be a literal tab in traditional `make`; this is notorious enough to deserve its own warning.

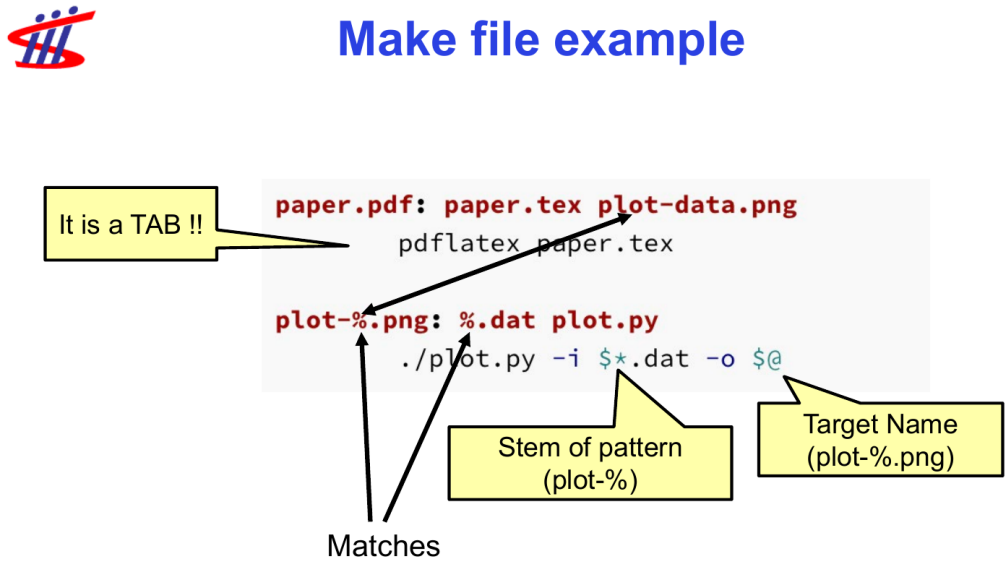


Figure 5: Pattern-rule example. The figure labels a target, a prerequisite pattern, and automatic variables such as `$@` (the current target) and `$*` (the stem matched by the pattern). It illustrates how `make` generalizes from one file to a family of similarly generated files.

Pattern rules make `make` much more expressive. A rule such as

Listing 21: A pattern rule for generated plot images.

```
plot-%.png: %.dat plot.py
    ./plot.py -i $*.dat -o $@
```

means that any file matching `plot-%.png` can be built from the correspondingly named `%.dat` input using the Python script. Here `$@` denotes the current target and `$*` denotes the matching stem.

11.3 Variables, clean targets, and repeated LaTeX builds

Variables improve readability and maintainability:

Listing 22: Variables and a phony cleanup rule.

```
NAME := main
TARGET := $(NAME)

all: $(TARGET).pdf

clean:
```

```
7       rm -f *.aux *.log *.out *.bbl *.blg $(TARGET).pdf
```

Notes that `:=` is often preferable to `=` because it performs immediate assignment instead of deferred expansion. This distinction becomes important when variables depend on other variables whose values may change later.

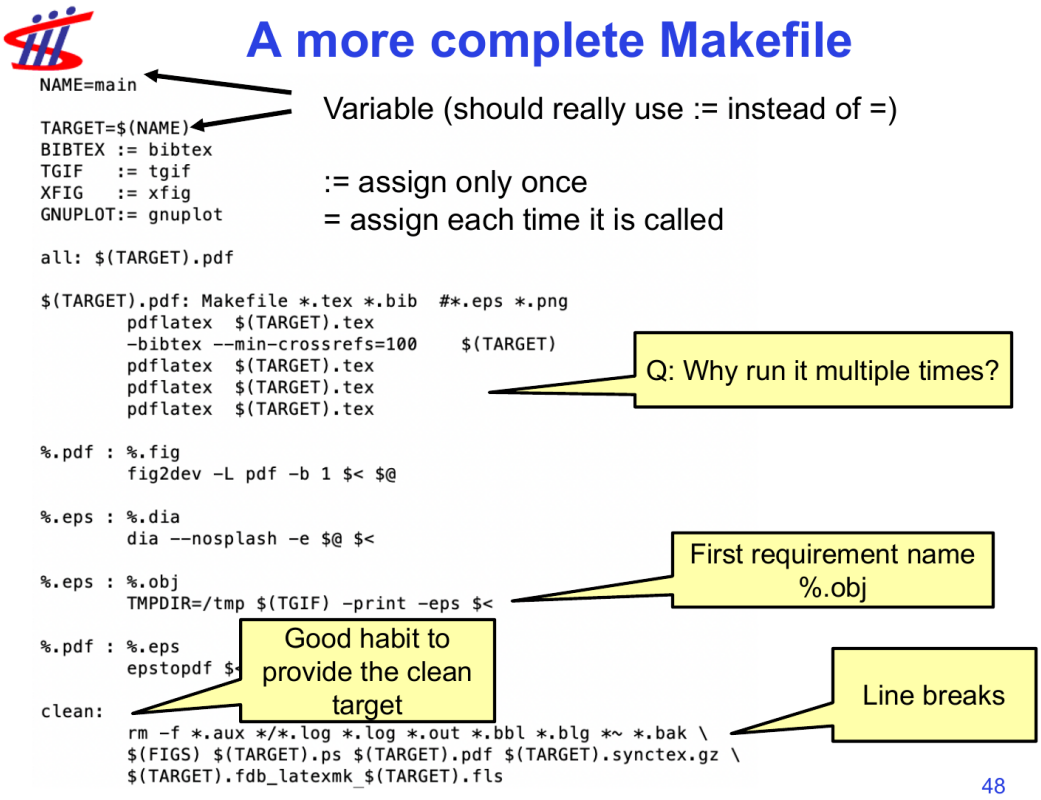


Figure 6: A more complete Makefile. The figure emphasizes immediate versus deferred variable assignment, pattern-based transformations, a conventional `clean` target, and the repeated execution of `pdflatex`. This is an excellent example of how build systems encode both dependency structure and tool-specific workflow knowledge.

Why does LaTeX often need to run multiple times? Because one pass writes cross-reference information, bibliography data, table-of-contents entries, and auxiliary files; later passes consume those files to resolve references correctly. Build automation captures such workflow knowledge so the user need not remember it manually.

---

12 Summary

The main ideas may be summarized as follows:

- The shell is a programmable interface to the operating system, not merely a place to type isolated commands.
- Files, terminals, pipes, and sockets share a common descriptor-based model, enabling elegant I/O redirection and inter-process composition.

- Environment variables and path lookup determine a large part of process behavior and are central to modern language tooling.
- Unix job control, signals, and remote-session tools matter because systems work is interactive, concurrent, and failure-prone.
- Globbing, regular expressions, and text-processing tools such as `find`, `sort`, `sed`, and `awk` form a practical language for working with large bodies of structured text.
- Build systems, shell scripts, and containers raise the level of abstraction from one-off commands to reproducible workflows.
- In the era of AI-assisted programming, understanding shell commands is increasingly important because we must be able to inspect, validate, and refine machine-generated operational instructions.

Recommended Follow-Up Reading

A highly relevant companion resource is *The Missing Semester of Your CS Education*, which develops many of the same ideas—shells, editors, version control, debugging, data wrangling, and remote machines—from a hands-on perspective.

13 Quizzes & Exercises

13.1 Short-Answer Questions

1. What is the core philosophy behind the design of Unix tools?
2. Explain the significance of the `root` user in a Linux system.
3. How do the operators `&&` and `||` differ when running multiple commands?
4. What are the three standard streams opened at the start of a process?
5. What does the `PATH` environment variable do, and how can it be modified?
6. Explain the function of the `tail -f` command.
7. Why is the `fork()` system call important for process inheritance?
8. What is the purpose of the `sudo` command?
9. Describe the difference between the signals sent by `Ctrl-C` and `Ctrl-Z`.
10. How does the `nohup` command assist with long-running tasks?

13.2 Essay Questions

1. **The Shell as a Connector:** Discuss how the Unix shell's design as an interpreted language using strings and lists facilitates inter-process communication. How has this design remained relevant for modern fields like AI?
2. **The Metaphor of Operating Systems as Airlines:** Analyze the operating-system "airline" jokes. What do these metaphors reveal about the perceived user experience and developer philosophy of Windows, macOS, and Linux?

3. **I/O Redirection and Data Flow:** Explain the mechanics of I/O redirection using pipes and file descriptors. How does this system allow a developer to create complex data-processing pipelines from simple, modular commands?
4. **Security and Access Control:** Discuss the balance between the power of the `root` user and the safety provided by standard user permissions and the `sudo` command. What are the risks and benefits of such a hierarchy?
5. **Process and Job Management:** Compare and contrast the tools available for monitoring and controlling system processes, such as `ps`, `top`, `bg`, and `kill`. How do these tools contribute to a developer's efficiency?

13.3 Selected Answer Sketches

1. Unix tools are designed to be small, composable, and focused. Their real power emerges when they are connected by the shell into larger pipelines or scripts.
2. The `root` user has effective UID 0 and bypasses many ordinary permission checks. This makes administrative tasks possible, but also makes mistakes unusually dangerous.
3. `&&` executes the second command only when the first succeeds, whereas `||` executes the second only when the first fails. Both rely on exit-status conventions.
4. The three standard streams are `stdin`, `stdout`, and `stderr`, corresponding to file descriptors 0, 1, and 2.
5. `PATH` tells the shell where to search for executables named without explicit path prefixes. It is commonly modified with assignments such as `export PATH=/new/dir:$PATH`.
6. `tail -f` prints the end of a file and continues following newly appended output. It is especially useful for logs or long-running program output redirected to a file.
7. Through `fork()`, a child process inherits descriptors and environment state from the parent. This is why new commands launched by a shell automatically inherit the terminal context and path environment.
8. `sudo` provides controlled privilege escalation for authorized users. It allows specific administrative commands to run as `root` without requiring a permanently privileged shell.
9. `Ctrl-C` usually sends an interrupt signal intended to terminate or interrupt the foreground process, while `Ctrl-Z` suspends it so the shell can manage it as a stopped job.
10. `nohup` causes a process to ignore hangup signals, allowing it to survive terminal disconnects that would otherwise terminate it.